

Conditional Flow-Matching Training

Trains a conditional diffusion model to learn $p(x_{t+1} \mid x_t)$.

What changes from the unconditional model:

- Dataset returns consecutive pairs (x_t, x_{t+1}) instead of single frames
- SongUNet uses `in_channels=4` (2 channels for noisy z_s + 2 channels for conditioning x_t concatenated spatially)
- x_t is passed as `class_labels` — concatenated inside `SongUNet.forward` before the encoder
- Training target is still $x_{t+1} - z_0$ (velocity), same formula as before
- `label_dropout=0.1` enables classifier-free guidance at inference if needed

Everything else (loss structure, optimiser, ODE sampler) is identical to the unconditional case.

1. Imports

```
In [ ]: import sys
        from pathlib import Path

        import numpy as np
        import torch
        import torch.optim as optim
        from torch.utils.data import DataLoader
        from tqdm import tqdm
        from natsort import natsorted
        import matplotlib.pyplot as plt
        import csv

        SQG_DIR = Path('SQG')
        sys.path.insert(0, str(SQG_DIR))

        from diffusion_networks import SongUNet
        from dataset import SQGPairDataset, DATA_1H, DATA_3H
        from autoregressive_forecasting import (
            sample_one_step,
            autoregressive_ensemble_rollout,
            compute_rmse,
            compute_ssr,
        )
```

2. Config

```
In [ ]: # — data —
        DATA_DIR = DATA_1H    # change to DATA_3H for 3-hour data
        DATA_STD = 2660.0      # normalisation std (mean assumed 0)

        # — model —
```

```

IMG_CHANNELS    = 2      # SQG has 2 vertical levels
IMG_RESOLUTION  = 64
FILTERS         = 32
LABEL_DROPOUT   = 0.1

# — training —
BATCH_SIZE      = 16
NUM_EPOCHS      = 20
LR              = 1e-3
WEIGHT_DECAY    = 1e-4

# — evaluation —
ROLLOUT_STEPS   = 10     # autoregressive steps to evaluate
N_ENSEMBLE      = 10     # ensemble members per rollout
ODE_STEPS       = 100    # Euler steps per ODE solve
N_INIT          = 20     # number of starting points to average metrics over
TRAIN_FRAC      = 0.8    # must match dataset split

# — misc —
SAVE_PATH = Path('../models/best_model_conditional.pth')
LOG_PATH  = Path('../models/training_log.csv')
SAVE_PATH.parent.mkdir(parents=True, exist_ok=True)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {device}')
print(f'Data:    {DATA_DIR}')

```

3. Dataset — consecutive pairs (x_t, x_{t+1})

`SQGPairDataset` is imported from `dataset.py`. Switch between 1-hour and 3-hour data by setting `DATA_DIR = DATA_1H` or `DATA_DIR = DATA_3H` in the config above.

```

In [4]: train_dataset = SQGPairDataset(DATA_DIR, std=DATA_STD, split='train')
        val_dataset   = SQGPairDataset(DATA_DIR, std=DATA_STD, split='val')

        train_loader  = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
        val_loader     = DataLoader(val_dataset,   batch_size=BATCH_SIZE, shuffle=False)

        print(f'Pairs — train: {len(train_dataset)}, val: {len(val_dataset)}')

```

Pairs — train: 8000, val: 2000

4. Model

The only architectural change from the unconditional model is `in_channels=4`.

Inside `SongUNet.forward`, when `class_labels` is provided it is concatenated channel-wise onto the noisy input `z_s` before the encoder:

`[z_s (2ch)] cat [x_t (2ch)] → [4ch input to encoder]`

The network therefore sees the full spatial structure of x_t at every location while learning the velocity toward x_{t+1} .

```
In [5]: model = SongUNet(
    img_resolution      = IMG_RESOLUTION,
    in_channels          = IMG_CHANNELS * 2,  # 2 (noisy z_s) + 2 (x_t cond
    out_channels         = IMG_CHANNELS,      # velocity has same shape as
    embedding_type       = 'fourier',
    encoder_type         = 'residual',
    decoder_type         = 'standard',
    channel_mult_noise   = 2,
    resample_filter      = [1, 3, 3, 1],
    model_channels       = FILTERS,
    channel_mult         = [2, 2, 2],
    attn_resolutions     = [32],
    label_dropout        = LABEL_DROPOUT,
).to(device)

print(f'Parameters: {sum(p.numel() for p in model.parameters()),}')

```

Parameters: 3,543,778

5. Flow-Matching Loss

Interpolant: $z_s = (1 - s) z_0 + s x_{t+1}$, $s \sim \text{Uniform}[0, 1]$, $z_0 \sim \mathcal{N}(0, I)$

Velocity target: $\dot{z}_s = x_{t+1} - z_0$ (constant along the linear path)

Loss: $\mathcal{L} = \mathbb{E}[\|\hat{b}(z_s, s, x_t) - (x_{t+1} - z_0)\|^2]$

The formula is identical to the unconditional case — the only difference is that the model now also receives x_t as spatial conditioning.

```
In [6]: def flow_matching_loss(model, x_t, x_t1):
    """
    Conditional flow-matching loss for one batch.

    Args:
        model : SongUNet with in_channels = 2*C
        x_t   : current state (B, C, H, W) — the conditioning input
        x_t1  : next state (B, C, H, W) — the generation target

    Returns:
        scalar loss
    """
    B = x_t1.shape[0]

    # sample interpolant time s ~ Uniform[0, 1]
    s = torch.rand(B, device=x_t1.device)  # (B,)
    s_spatial = s.view(B, 1, 1, 1)         # broadcast over

    # sample base noise
    z0 = torch.randn_like(x_t1)             # (B, C, H, W)

    # form interpolated noisy sample at time s
    z_s = (1 - s_spatial) * z0 + s_spatial * x_t1  # (B, C, H, W)

    # target velocity (same at every s along the linear path)
    target = x_t1 - z0                       # (B, C, H, W)

```

```

# model prediction
# noise_labels = s (interpolant time plays the role of noise level)
# class_labels = x_t (spatially concatenated inside SongUNet before t
pred = model(z_s, s, class_labels=x_t) # (B, C, H, W)

return ((pred - target) ** 2).mean()

```

6. Optimiser

```

In [7]: optimizer = optim.AdamW(model.parameters(), lr=LR, weight_decay=WEIGHT_DE

# cosine decay: learning rate decreases smoothly over training
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=NUM_EPO

# linear warmup for the first 500 steps: avoids large unstable updates at
warmup = optim.lr_scheduler.LinearLR(optimizer, start_factor=0.01, end_fa

```

7. Training Loop

```

In [7]: train_losses, val_losses = [], []
best_val_loss = float('inf')

with open(LOG_PATH, 'w', newline='') as f:
    csv.writer(f).writerow(['epoch', 'train_loss', 'val_loss'])

for epoch in range(NUM_EPOCHS):

    # — training phase —————
    model.train()
    total_train = 0.0
    for x_t, x_t1 in tqdm(train_loader, desc=f'Epoch {epoch+1}/{NUM_EPOCH
        x_t = x_t.to(device)
        x_t1 = x_t1.to(device)

        optimizer.zero_grad()
        loss = flow_matching_loss(model, x_t, x_t1)
        loss.backward()
        optimizer.step()
        warmup.step() # warmup steps per batch, not per epoch

        total_train += loss.item()

    avg_train = total_train / len(train_loader)

    # — validation phase —————
    model.eval()
    total_val = 0.0
    with torch.no_grad():
        for x_t, x_t1 in val_loader:
            x_t = x_t.to(device)
            x_t1 = x_t1.to(device)
            total_val += flow_matching_loss(model, x_t, x_t1).item()

    avg_val = total_val / len(val_loader)

    # — bookkeeping —————
    train_losses.append(avg_train)

```

```

val_losses.append(avg_val)
scheduler.step()  # cosine decay steps per epoch

if avg_val < best_val_loss:
    best_val_loss = avg_val
    torch.save(model.state_dict(), SAVE_PATH)
    tag = ' <- best'
else:
    tag = ''

with open(LOG_PATH, 'a', newline='') as f:
    csv.writer(f).writerow([epoch + 1, avg_train, avg_val])

print(f'Epoch {epoch+1:3d}  train={avg_train:.4f}  val={avg_val:.4f}{
print(f'\nTraining done. Best val loss: {best_val_loss:.4f}')

```

```

Epoch   1  train=0.5608  val=0.1119  <- best
Epoch   2  train=0.1763  val=0.0759  <- best
Epoch   3  train=0.1551  val=0.0683  <- best
Epoch   4  train=0.1433  val=0.0716
Epoch   5  train=0.1386  val=0.0645  <- best
Epoch   6  train=0.1317  val=0.0527  <- best
Epoch   7  train=0.1307  val=0.0534
Epoch   8  train=0.1312  val=0.0435  <- best
Epoch   9  train=0.1270  val=0.0413  <- best
Epoch  10  train=0.1216  val=0.0404  <- best
Epoch  11  train=0.1189  val=0.0492
Epoch  12  train=0.1180  val=0.0545
Epoch  13  train=0.1150  val=0.0353  <- best
Epoch  14  train=0.1186  val=0.0368
Epoch  15  train=0.1140  val=0.0493
Epoch  16  train=0.1149  val=0.0431
Epoch  17  train=0.1147  val=0.0383
Epoch  18  train=0.1089  val=0.0405
Epoch  19  train=0.1098  val=0.0343  <- best

```

Epoch	20	train=0.1106	val=0.0380	
Epoch	21	train=0.1082	val=0.0358	
Epoch	22	train=0.1060	val=0.0416	
Epoch	23	train=0.1044	val=0.0329	<- best
Epoch	24	train=0.1043	val=0.0291	<- best
Epoch	25	train=0.1055	val=0.0335	
Epoch	26	train=0.1035	val=0.0322	
Epoch	27	train=0.1088	val=0.0287	<- best
Epoch	28	train=0.1009	val=0.0332	
Epoch	29	train=0.0995	val=0.0269	<- best
Epoch	30	train=0.0992	val=0.0258	<- best
Epoch	31	train=0.0951	val=0.0297	
Epoch	32	train=0.1035	val=0.0281	
Epoch	33	train=0.1038	val=0.0276	
Epoch	34	train=0.0980	val=0.0265	
Epoch	35	train=0.1016	val=0.0233	<- best
Epoch	36	train=0.0996	val=0.0256	
Epoch	37	train=0.0988	val=0.0311	
Epoch	38	train=0.0938	val=0.0237	
Epoch	39	train=0.0949	val=0.0247	
Epoch	40	train=0.0944	val=0.0260	
Epoch	41	train=0.0954	val=0.0244	
Epoch	42	train=0.0945	val=0.0254	
Epoch	43	train=0.0919	val=0.0209	<- best
Epoch	44	train=0.0915	val=0.0264	
Epoch	45	train=0.0885	val=0.0231	

```

Epoch 46 train=0.0889 val=0.0245

Epoch 47 train=0.0930 val=0.0237

Epoch 48 train=0.0916 val=0.0211

Epoch 49 train=0.0949 val=0.0191 <- best

Epoch 50 train=0.0987 val=0.0214

Training done. Best val loss: 0.0191

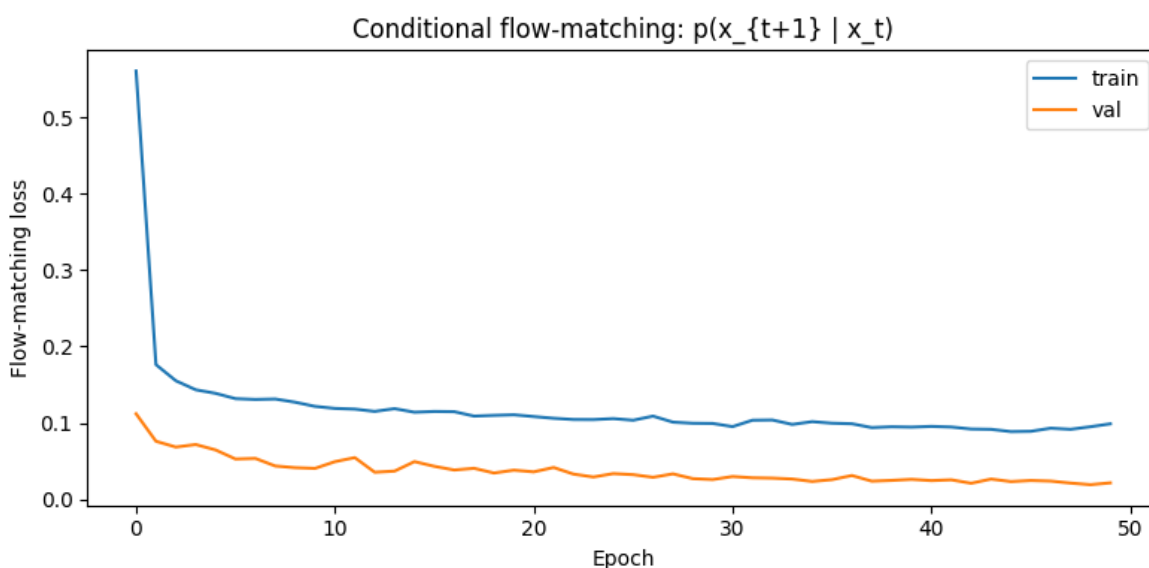
```

8. Loss Curves

```

In [8]: plt.figure(figsize=(8, 4))
plt.plot(train_losses, label='train')
plt.plot(val_losses, label='val')
plt.xlabel('Epoch')
plt.ylabel('Flow-matching loss')
plt.title('Conditional flow-matching:  $p(x_{t+1} | x_t)$ ')
plt.legend()
plt.tight_layout()
plt.show()

```



9. Sanity Check — sample one forecast

Given conditioning state x_t , integrate the flow ODE from noise ($s = 0$) to data ($s = 1$) to get predicted $\hat{x}_{t+1}$, then compare with the ground truth.

```

In [ ]: # load best checkpoint
model.load_state_dict(torch.load(SAVE_PATH, map_location=device))

x_t_sample, x_t1_truth = next(iter(val_loader))
x_t_sample = x_t_sample[:4].to(device)
x_t1_truth = x_t1_truth[:4]

```

```

# sample_one_step from autoregressive_forecasting (no duplicate implement
x_t1_pred = sample_one_step(model, x_t_sample, steps=ODE_STEPS).cpu()

fig, axes = plt.subplots(3, 4, figsize=(12, 9))
for i in range(4):
    axes[0, i].imshow(x_t_sample[i, 0].cpu(), cmap='RdBu_r')
    axes[0, i].set_title(f'x_t (cond) #{i}')
    axes[1, i].imshow(x_t1_pred[i, 0], cmap='RdBu_r')
    axes[1, i].set_title('x_{t+1} (pred)')
    axes[2, i].imshow(x_t1_truth[i, 0], cmap='RdBu_r')
    axes[2, i].set_title('x_{t+1} (truth)')
    for row in axes:
        row[i].axis('off')

plt.suptitle('Conditional forecast sanity check (level 0)', y=1.01)
plt.tight_layout()
plt.show()

```

10. Autoregressive Rollout Evaluation

Rolls out the ensemble autoregressively for `ROLLOUT_STEPS=10` and reports **RMSE** and **SSR** at every lead time, averaged over `N_INIT` validation starting points.

```

In [ ]: # Load val trajectories (full sequences, not pairs) for rollout evaluation
files = natsorted(DATA_DIR.glob('sqq_N64_*hrly_*.npz'))
cut = int(len(files) * TRAIN_FRAC)
val_files = files[cut:]

val_trajs = []
for f in val_files:
    arr = torch.tensor(np.load(f).astype(np.float32) / DATA_STD) # (T, C)
    val_trajs.append(arr)

T_per_traj = val_trajs[0].shape[0]
print(f'Val trajectories: {len(val_trajs)}, timesteps: {T_per_traj}')

# Fixed init points (seed=42) - ensures t + ROLLOUT_STEPS < T
max_t = T_per_traj - ROLLOUT_STEPS - 1
rng = np.random.default_rng(42)
traj_indices = rng.integers(0, len(val_trajs), size=N_INIT)
time_indices = rng.integers(0, max_t, size=N_INIT)
init_points = list(zip(traj_indices.tolist(), time_indices.tolist()))
print(f'Sampled {N_INIT} init points')

```

```

In [ ]: all_rmse, all_ssr = [], []

for ti, t in tqdm(init_points, desc='Evaluating init points'):
    x0 = val_trajs[ti][t] # (C, H, W)
    truth = val_trajs[ti][t + 1 : t + 1 + ROLLOUT_STEPS].numpy() # (steps, C, H, W)

    preds = autoregressive_ensemble_rollout(
        model, x0, ROLLOUT_STEPS, N_ENSEMBLE, ODE_STEPS, device=device
    ) # (steps, n_ens, C, H, W)

    preds_mean = preds.mean(axis=1) # (steps, C, H, W)
    rmse = compute_rmse(preds_mean, truth) # (steps,)
    ssr = compute_ssr(preds, rmse) # (steps,)

```



```
all_rmse.append(rmse)
all_ssr.append(ssr)

mean_rmse = np.stack(all_rmse).mean(axis=0) # (steps,)
mean_ssr = np.stack(all_ssr).mean(axis=0)
lead_times = np.arange(1, ROLLOUT_STEPS + 1)

print(f'\nAutoregressive rollout results (label_dropout={LABEL_DROPOUT},
print(f'{"Lead":>5} {"RMSE":>8} {"SSR":>8}')
for t, r, s in zip(lead_times, mean_rmse, mean_ssr):
    print(f'{int(t):5d} {r:8.4f} {s:8.4f}')
```

```
In [ ]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

ax1.plot(lead_times, mean_rmse, marker='o', color='steelblue')
ax1.set_xlabel('Lead time (steps)')
ax1.set_ylabel('RMSE (normalised units)')
ax1.set_title('RMSE vs Lead Time')
ax1.grid(True, alpha=0.3)

ax2.plot(lead_times, mean_ssr, marker='o', color='seagreen')
ax2.axhline(1.0, color='red', linestyle='--', label='ideal SSR = 1')
ax2.set_xlabel('Lead time (steps)')
ax2.set_ylabel('SSR')
ax2.set_title('Spread/Skill Ratio vs Lead Time')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.suptitle(f'Autoregressive Rollout | label_dropout={LABEL_DROPOUT},
plt.tight_layout()
plt.show()
```